中山大学计算机学院

人工智能实验报告

课程名称：Artificial Intelligence

教学班级	超算班	专业	信息与计算科学
学号	21307261	姓名	王健阳

一，实验题目

博弈树搜索

二，实验内容

算法原理

根据当前的状态（棋局）最为博弈树的max根节点，借助dfs去遍历。

递归定义：

对于某个棋局：

1. 此棋局已经分出胜负了，根据胜负方直接返回正负无穷
2. 当到达最大深度时停止继续向下搜索，返回当前棋局的评估数值。
3. 遍历当前棋局的所有可能走法，并再次递归调用ab函数（切换极大极小层），记录下每个走法对应的棋局评估值。若对于max节点，评估值大于等于函数参数beta的时候可以作为返回值；对于min节点，评估值小于等于函数参数alpha的时候可以作为返回值。

伪代码

剪枝算法的大致流程图：

（此处伪代码通过取反再取反的方式，使得对于ab函数而言，每一层都可以当做max层处理，避免了分类）

```
int AlphaBeta(int depth, int alpha, int beta) {
    if (depth == 0) {
        return Evaluate();
    }
    GenerateLegalMoves();
    while (MovesLeft()) {
        MakeNextMove();
        val = -AlphaBeta(depth - 1, -beta, -alpha);
        UnmakeMove();
        if (val >= beta) {
            return beta;
        }
        if (val > alpha) {
            alpha = val;
        }
    }
    return alpha;
}
```

关键代码展示

```
    def alpha_beta(self, depth, alpha, beta):
        who = (self.max_depth - depth) % 2
        # 判断是否游戏结束
        if self.game_over(who):
            return cc.min_val
        # 搜索到指定深度了，直接返回当前的评估值
        if depth == 1:
            return self.evaluate(who)
        # 返回当前棋局所有可能的走法
        move_list = self.chessboard.PossibleMove(who)

        # 获取每个走法对应的历史分数，初始化为全0
        for i in range(len(move_list)):
            move_list[i].score = self.history_table.GetHistoryScore(who,
move_list[i])
        # 历史分数最小的将被优先遍历
        # 总遍历深度：将每次遍历的深度累加
        # 故最优先遍历的是那些从未到达过的棋局，然后依次按照总遍历深度的次序遍历
        move_list.sort()
        best_step = move_list[0]
        for step in move_list:
            temp = self.move_to(step)
            # 通过去负号来避免分类讨论，等于每一层都是最大层（相对who而言）
            score = -self.alpha_beta(depth - 1, -beta, -alpha)
            self.move_back(step, temp)

            if score >= beta:
                return beta
            if score > alpha:
```

```
                alpha = score
                # 记录生成博弈树第一层的对应最佳走法作为AI下一步的走法
                if depth == self.max_depth:
                    self.best_move = step
                # 此时的得分已经超过了alpha，故更新alpha，同时当前走法是目前的最优解
                best_step = step
            '''
            if score > alpha:
                alpha = score
                # 记录生成博弈树第一层的对应最佳走法作为AI下一步的走法
                if depth == self.max_depth:
                    self.best_move = step
                best_step = step
            if alpha >= beta:
                best_step = step
                return beta
            '''
        # 更新历史表，如果best_step非空
        if best_step.from_x != -1:
            # 历史分数加上当前的深度值
            self.history_table.AddHistoryScore(who, best_step, depth)
        return alpha
```

创新点 & 优化

> 空间优化（数据结构优化）

当需要获取某棋局所有可能的邻接棋局的时候，存储的不是棋局本身而是走法。这样子做可以大大降低算法的空间时间复杂度，对比如下：

走法类

```
class Step:  # 走法类
    def __init__(self, from_x=-1, from_y=-1, to_x=-1, to_y=-1):
        self.from_x = from_x
        self.from_y = from_y
        self.to_x = to_x
        self.to_y = to_y
        self.score = 0

    def __lt__(self, other):
        return self.score > other.score

    def __eq__(self, other):
        return self.score == other.score

    def __str__(self):
        return 'steps: ' + '(' + str(self.from_x) + ', ' \
               + str(self.from_y) + ') -> (' + str(self.to_x) \
               + ', ' + str(self.to_y) + ')'
```

棋局类（部分）

```
class ChessBoard:
    def __init__(self):
        self.board = []  # 棋盘
        for i in range(9):
            self.board.append([])
            for j in range(10):
                if cc.initial_chessboard[i][j] == 0:
                    self.board[i].append(ChessProperty(-1,
cc.initial_chessboard[i][j]))
                else:
                    self.board[i].append(ChessProperty(0 if j < 5 else 1,
cc.initial_chessboard[i][j]))
```

故算法进行时实际上只有一个“动态棋盘”，其棋子的移动受算法类控制。对于dfs的每一次回溯，还具有放回棋子（悔棋）的功能

> 剪枝速度优化

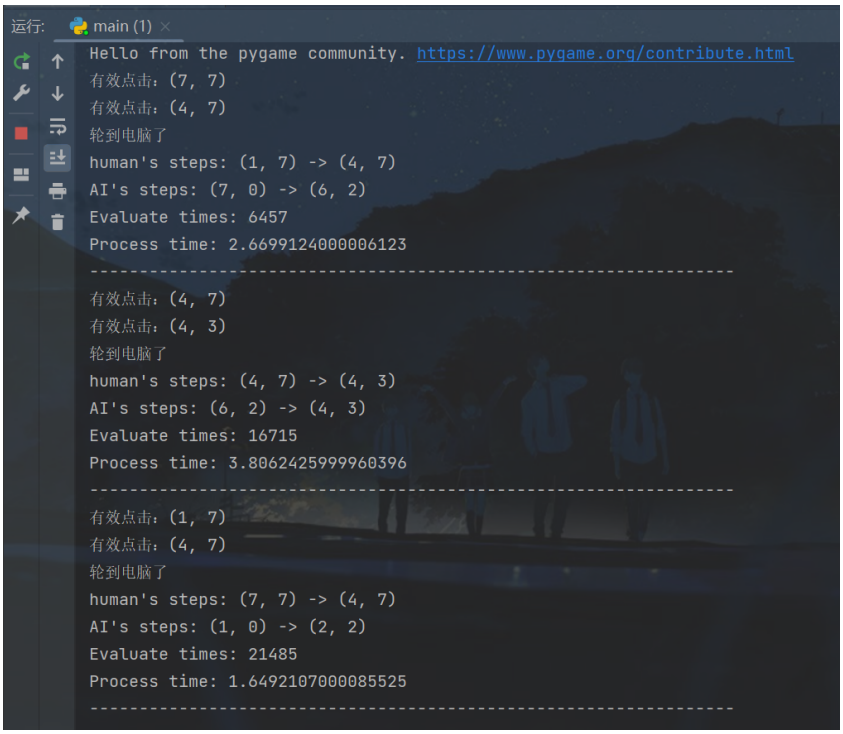
采用一个历史表来存储所有走法的已知信息。对于9*10的象棋棋盘，一共有2*90*90种可能的走法。对每一种走法设立初始评估值0，在ab剪枝的过程中不断刷新。因为在暴力搜索的时候，绝大多数都是相同的走法不断地去重复（例如你一次只可以下一枚棋子，其余所有棋子的走法在相邻二局内几乎无任何变化）。对每一种走法给予一个取决于当前深度的分数，并在遍历前按照从小到大的顺序遍历可能的走法。

即对于某一个走法，其被访问的次数越多，ab第三步遍历的次序就越靠后。使得整个算法dfs的时候更加有目的性。

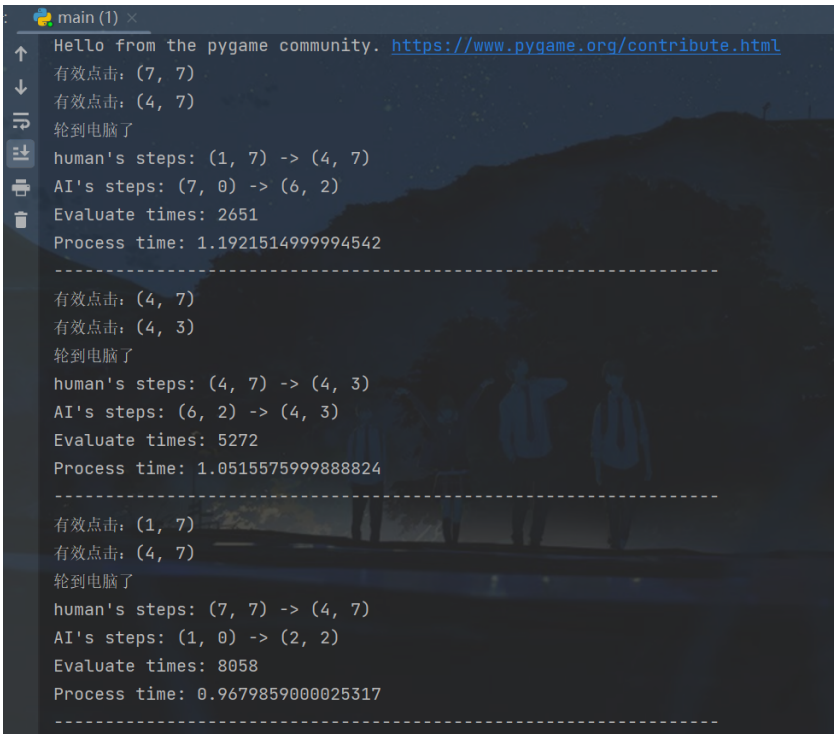
`Evaluate times`：评估次数

`Pcocess time`：计算时间

优化前：



优化后：



> 评估函数优化

在网上找到的一个较好的评估函数，从四个角度进行评估：

1. 棋子自身的属性价值（不同棋子分数不一）
2. 棋子的位置价值（仅取决于棋子自身位置）
3. 棋子的机动性价值（取决于可移动到的空位置）
4. 棋子的关系价值（走出此步之后的攻击，被攻击，保护，被保护情况）

关系价值最为关键，体现出AI是否有必要跟你进行换子（如一换一，二换三等）

```
    def evaluate(self, who):
        self.cnt += 1
        relation_list = self.relation_list_init()
        base_val = [0, 0]
        pos_val = [0, 0]
        mobile_val = [0, 0]
        relation_val = [0, 0]
        for x in range(9):
            for y in range(10):
                now_chess = self.chessboard.board[x][y]
                _type = now_chess.chess_type
                if _type == 0:
                    continue
                # now = 0 if who else 1
                now = now_chess.belong
                pos = x * 9 + y
                temp_move_list = self.chessboard.GetChessMove(x, y, now, True)
                # 计算基础价值
                base_val[now] += cc.base_val[_type]
                # 计算位置价值
                if now == 0:
                    # 如果是要求最大值的玩家
                    pos_val[now] += cc.pos_val[_type][pos]
                else:
                    pos_val[now] += cc.pos_val[_type][89 - pos]
                # 计算机动性价值，记录关系信息
                for item in temp_move_list:
                    # 目的位置的棋子
                    temp_chess = self.chessboard.board[item.to_x][item.to_y]
                    if temp_chess.chess_type == cc.nul:
                        # 如果是空，那么加上机动性值
                        mobile_val[now] += cc.mobile_val[_type]
                        continue
                    elif temp_chess.belong != now:
                        # 如果不是自己一方的棋子
                        if temp_chess.chess_type == cc.kng:
                            # 如果能吃了对方的将，那么就赢了
                            if temp_chess.belong != who:
                                return cc.max_val
                            else:
                                relation_val[1 - now] -= 20
                                # 如果不能，那么就相当于被将军，对方要减分
                                continue
                        # 记录now_chess攻击了谁
                        relation_list[x][y].attack[relation_list[x][y].num_attack]
= temp_chess.chess_type
                        relation_list[x][y].num_attack += 1
                        relation_list[item.to_x][item.to_y].chess_type =
temp_chess.chess_type
                        # 记录temp_chess被谁攻击
                        relation_list[item.to_x][item.to_y].attacked[
```

```
                                relation_list[item.to_x][item.to_y].num_attacked] =
_type
                            relation_list[item.to_x][item.to_y].num_attacked += 1
                        elif temp_chess.belong == now:
                            if temp_chess.chess_type == cc.kng:  # 保护自己的将没有意
义，直接跳过
                                continue
                            # 记录now_chess保护了谁
                            relation_list[x][y].guard[relation_list[x][y].num_guard] =
temp_chess
                            relation_list[x][y].num_guard += 1
                            # 记录temp_chess被谁所保护
                            relation_list[item.to_x][item.to_y].chess_type =
temp_chess.chess_type
                            relation_list[item.to_x][item.to_y].guarded[
                                relation_list[item.to_x][item.to_y].num_guarded] =
_type
                            relation_list[item.to_x][item.to_y].num_guarded += 1
        for x in range(9):
            for y in range(10):
                num_attacked = relation_list[x][y].num_attacked
                num_guarded = relation_list[x][y].num_guarded
                now_chess = self.chessboard.board[x][y]
                _type = now_chess.chess_type
                now = now_chess.belong
                unit_val = cc.base_val[now_chess.chess_type] >> 3
                sum_attack = 0  # 被攻击总子力
                sum_guard = 0
                min_attack = 999  # 最小的攻击者
                max_attack = 0  # 最大的攻击者
                max_guard = 0
                flag = 999  # 有没有比这个子的子力小的
                if _type == cc.nul:
                    continue
                # 统计攻击方的子力
                for i in range(num_attacked):
                    temp = cc.base_val[relation_list[x][y].attacked[i]]
                    flag = min(flag, min(temp, cc.base_val[_type]))
                    min_attack = min(min_attack, temp)
                    max_attack = max(max_attack, temp)
                    sum_attack += temp
                # 统计防守方的子力
                for i in range(num_guarded):
                    temp = cc.base_val[relation_list[x][y].guarded[i]]
                    max_guard = max(max_guard, temp)
                    sum_guard += temp
                if num_attacked == 0:
                    relation_val[now] += 5 * relation_list[x][y].num_guarded
                else:
                    multi_val = 5 if who != now else 1
                    if num_guarded == 0:
                        # 如果没有保护
                        relation_val[now] -= multi_val * unit_val
                    else:
```

```
                        # 如果有保护
                        if flag != 999:
                            # 存在攻击者子力小于被攻击者子力,对方将愿意换子
                            relation_val[now] -= multi_val * unit_val
                            relation_val[1 - now] -= multi_val * (flag >> 3)
                        # 如果是二换一，并且最小子力小于被攻击者子力与保护者子力之和，
则对方可能以一子换两子
                        elif num_guarded == 1 and num_attacked > 1 and \
                                min_attack < cc.base_val[_type] + sum_guard:
                            relation_val[now] -= multi_val * unit_val
                            relation_val[now] -= multi_val * (sum_guard >> 3)
                            relation_val[1 - now] -= multi_val * (flag >> 3)
                        # 如果是三换二并且攻击者子力较小的二者之和小于被攻击者子力与保
护者子力之和,则对方可能以两子换三子
                        elif num_guarded == 2 and num_attacked == 3 and \
                                sum_attack - max_attack < cc.base_val[_type] +
sum_guard:
                            relation_val[now] -= multi_val * unit_val
                            relation_val[now] -= multi_val * (sum_guard >> 3)
                            relation_val[1 - now] -= multi_val * ((sum_attack -
max_attack) >> 3)
                        # 如果是n换n，攻击方与保护方数量相同并且攻击者子力小于被攻击者
子力与保护者子力之和再减去保护者中最大子力,则对方可能以n子换n子
                        elif num_guarded == num_attacked and \
                                sum_attack < cc.base_val[now_chess.chess_type] +
sum_guard - max_guard:
                            relation_val[now] -= multi_val * unit_val
                            relation_val[now] -= multi_val * ((sum_guard -
max_guard) >> 3)
                            relation_val[1 - now] -= sum_attack >> 3
        my_max_val = base_val[0] + pos_val[0] + mobile_val[0] + relation_val[0]
        my_min_val = base_val[1] + pos_val[1] + mobile_val[1] + relation_val[1]
        if who == 0:
            return my_max_val - my_min_val
        else:
            return my_min_val - my_max_val
```

三，实验结果及分析

算法结果展示实例

视频展示在result中

> 初始化界面展示

> 对局部分截图展示

评测指标展示及分析

此次实验的评测指标主要是：AI的计算时间，AI每一步的针对性强弱。这些都在视频中有所体现

四，思考题

此次实验无思考题

五，参考资料

1.基本搜索方法——Alpha-Beta搜索 (xqbase.com)

2.SYSU_AI_lab/lab5_alpha-beta剪枝中国象棋/[CS]_20337025_cuicanming/code at main · 91Mrcui/SYSU_AI_lab · GitHub

3.(47条消息) 中国象棋python实现(拥有完整源代码) Alpha-beta剪枝+GUI+历史启发式+有普通人棋力_象棋人机对战程序_Veritaswhs的博客-CSDN博客

界面的设计参考了3，算法的流程借鉴了三者的思路。由于2提供了一个完整项目，故在逐渐理清了其2000多行的代码组织之后，以自己的方式将代码重新梳理了一遍，以自己的理解修改了一些界面细节，并试着按照1中的流程图重写了部分核心ab代码（注释部分为原代码，两者的描述略有不同）。